

1. Index Fundamentals

- What is an index?
- Why indexes improve performance
- Trade-offs (read vs write cost)
- Basic concept of how DB searches data

1.1 What Is an Index?

An **index** is a separate data structure that PostgreSQL maintains alongside your table. It stores **sorted values** from one or more columns paired with **pointers (TIDs)** to the actual rows in the heap, enabling the database to find data without reading every row.

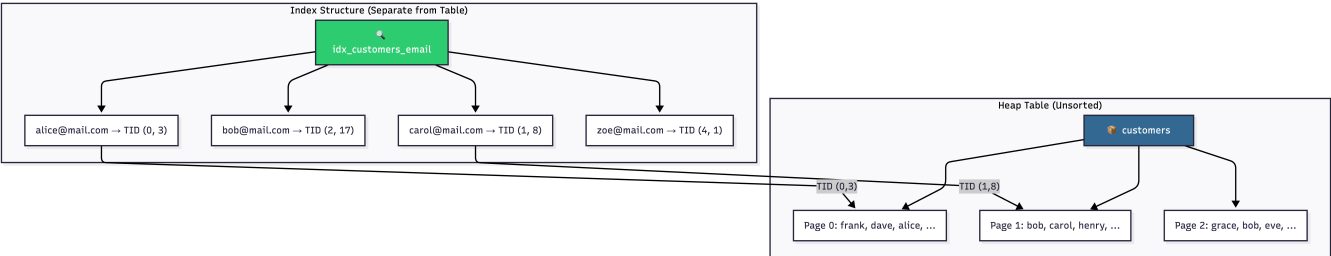
[blocked URL](#)

Analogy

Think of a phone book — names are sorted alphabetically, so you can jump straight to "Smith" instead of reading every entry from page one. A database index works the same way for your table columns.

Creating a simple index

```
-- Creating an index
CREATE INDEX idx_customers_email ON customers (email);
```



Key Facts

Property	Detail
Stored separately	Index is its own file on disk, not part of the table
Points to heap	Each entry contains a TID (page, offset) pointing to the row
Sorted	Entries are ordered (in a B-Tree), unlike the heap
Maintained automatically	PostgreSQL updates indexes on every INSERT, UPDATE, DELETE
Multiple per table	You can have many indexes on one table

1.2 Basic Concept of How PostgreSQL Searches Data

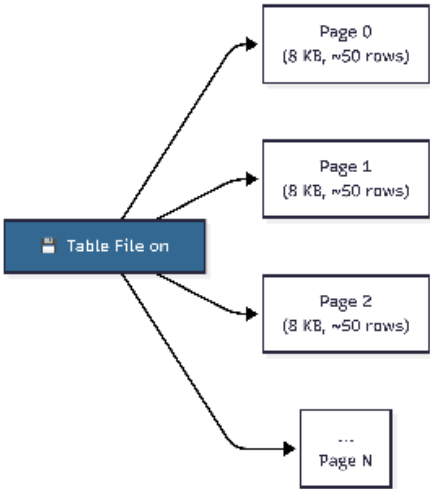
Databases don't read individual rows from disk. Data is stored in fixed-size blocks called **pages** (typically **8 KB** in SQL Server, **8 KB** in PostgreSQL). Every read or write operation happens at the page level.

Database	Page Size	Terminology
PostgreSQL	8 KB	Pages (grouped into 1 GB segments)
SQL Server	8 KB	Pages (grouped into 64 KB extents)
MySQL/InnoDB	16 KB	Pages



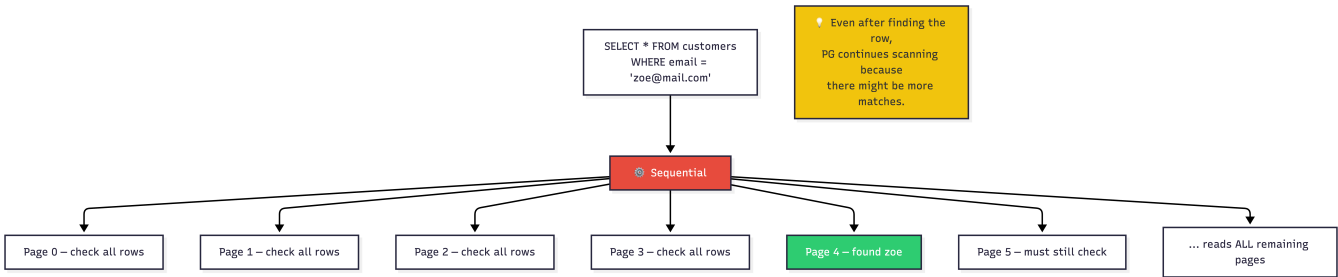
Key Insight:

Performance = minimizing the number of **page I/O** operations. Indexes exist to reduce page reads.



Without an Index — Sequential Scan

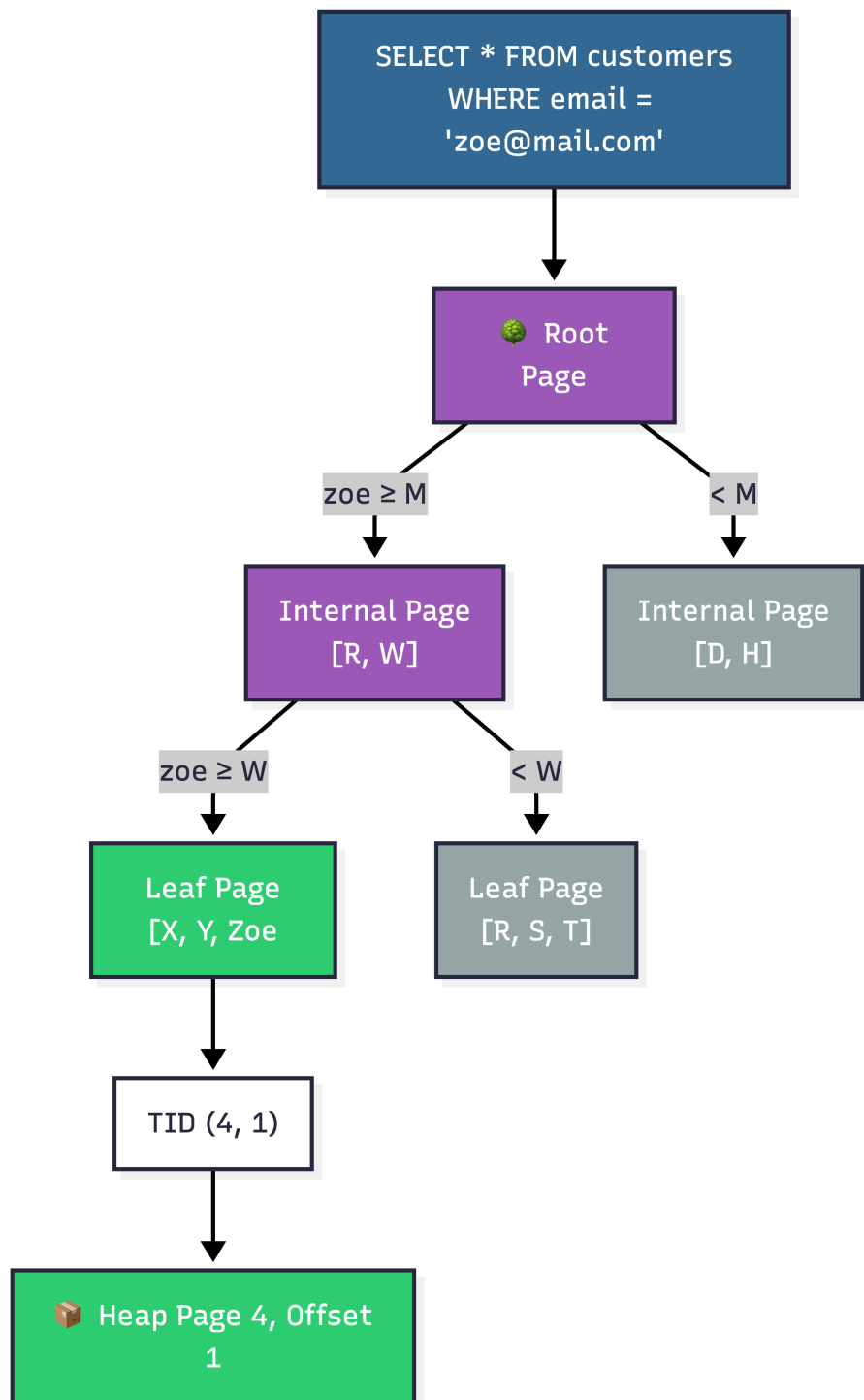
PostgreSQL reads **every page** from first to last, checking each row against your WHERE condition.



Cost: Reads ALL pages $O(N)$ where N = total pages. For a 10M row table **200,000 page reads**.

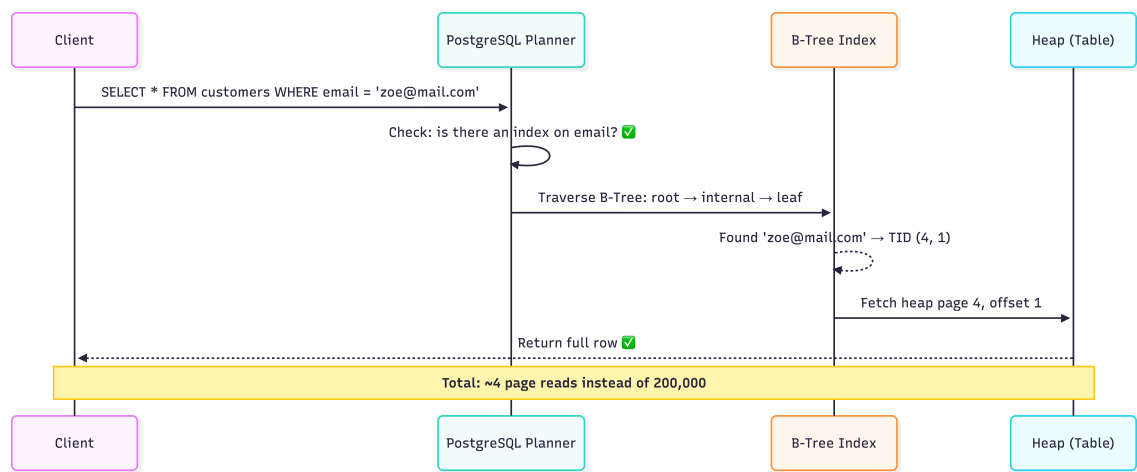
With an Index — B-Tree Lookup

PostgreSQL navigates a **balanced tree**. At each level, it eliminates half (or more) of the remaining entries.



Cost: Reads only **3–5 pages** (tree depth + 1 heap page) $O(\log N)$. For 10M rows **4–5 page reads**.

The Complete Search Process



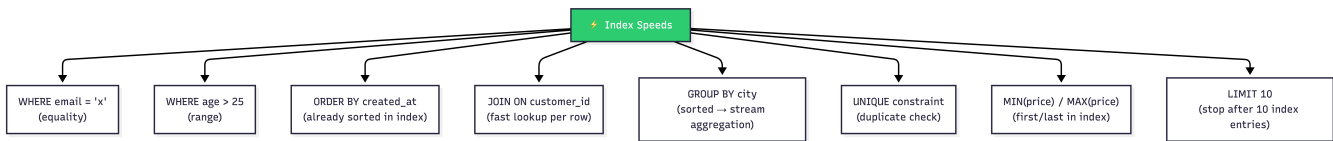
Time Complexity: $O(\log N)$ — for 10 million rows, only ~23 page reads instead of hundreds of thousands.

1.3 Why Indexes Improve Performance

Table Size	Pages (~50 rows/page)	Seq Scan (pages read)	Index Lookup (pages read)	Speedup
1,000 rows	20	20	2–3	8×
100,000 rows	2,000	2,000	3–4	600×
10,000,000 rows	200,000	200,000	4–5	45,000×
1 billion rows	20,000,000	20,000,000	5–6	3,600,000×

A B-Tree with 10 million entries is only ~4 levels deep. Each level = 1 page read.

What Operations Benefit



Practical Proof

```
-- Without index:
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customers WHERE email = 'zoe@mail.com';
-- Seq Scan on customers (actual time=847.32..847.33 rows=1)
-- Buffers: shared read=185000 185,000 page reads!

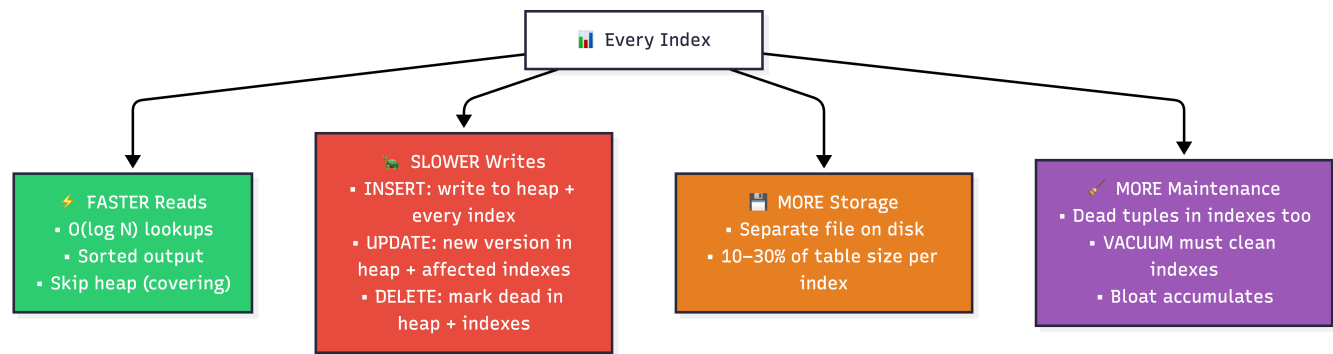
-- Create index:
CREATE INDEX idx_customers_email ON customers (email);

-- With index:
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customers WHERE email = 'zoe@mail.com';
-- Index Scan using idx_customers_email (actual time=0.03..0.04 rows=1)
-- Buffers: shared hit=4 4 page reads!
```

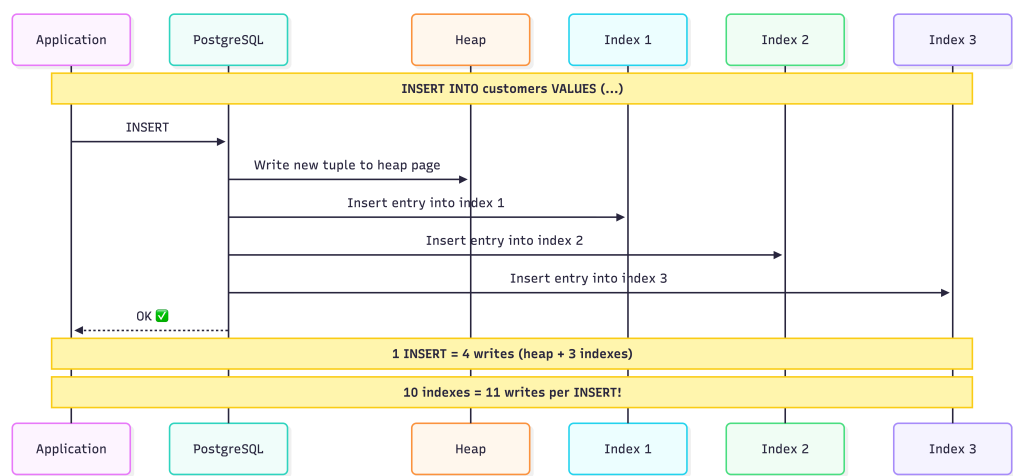
1.4 Trade-offs: Read vs Write Cost

Indexes are **not free**. Every index you add comes with ongoing costs.

The Trade-off Diagram



Write Cost Breakdown

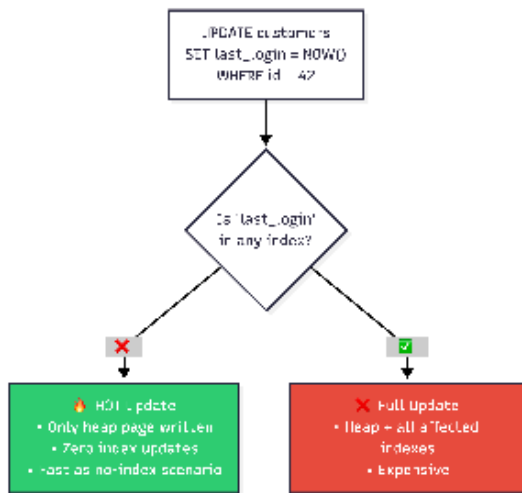


The Cost Matrix

Operation	0 Indexes	1 Index	5 Indexes	10 Indexes
SELECT(filtered)	Seq Scan	Index Scan	Index Scan	Index Scan
INSERT	1 write	2 writes	6 writes	11 writes
UPDATE(indexed col)	1 write	2 writes	2–6 writes	2–11 writes
UPDATE (non-indexed col)	1 write	1 write (HOT)	1 write (HOT)	1 write (HOT)
Storage	baseline	+15%	+60%	+120%

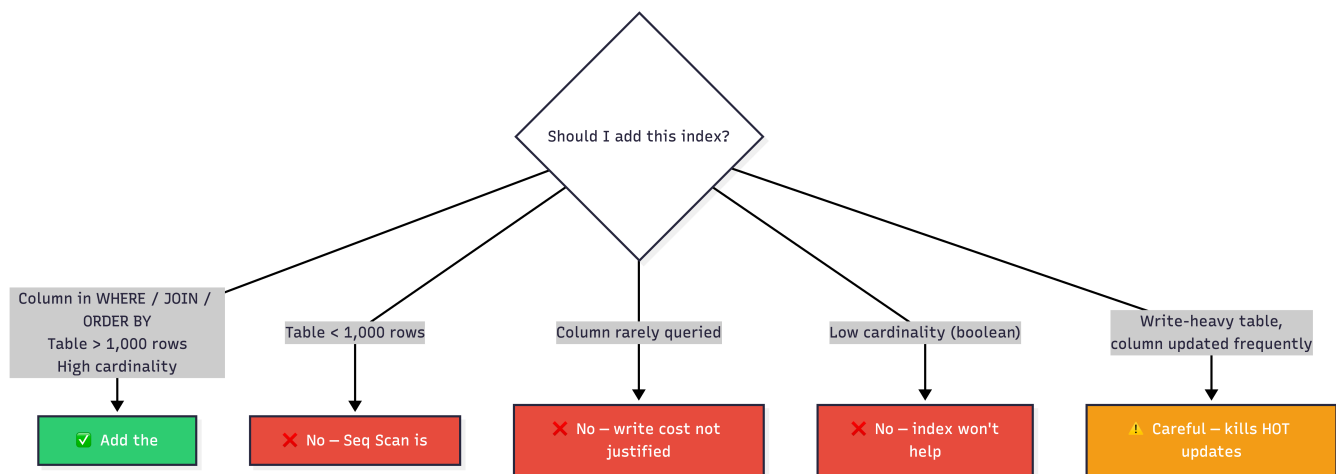
HOT Updates — PostgreSQL's Safety Valve

If your UPDATE **does not modify any indexed column** and the new row fits on the **same page**, PostgreSQL performs a **HOT (Heap-Only Tuple) update** — it skips all index updates entirely.



Lesson: Don't index columns that are frequently updated but rarely searched (e.g., `last_login`, `updated_at`, `view_count`).

When to Index vs When Not To



Quick Reference

Concept	One-Line Summary
Index	Sorted lookup structure with pointers (TIDs) to heap rows
Heap	The actual table — rows stored in no particular order
Page	8 KB block — the smallest unit PostgreSQL reads/writes
TID	Tuple Identifier (page, offset) — an index entry's pointer to a row
Seq Scan	Read all pages linearly — $O(N)$
Index Scan	Navigate B-Tree then fetch from heap — $O(\log N)$
HOT Update	Skips index maintenance when indexed columns unchanged
Trade-off	Every index = faster reads, slower writes, more storage, more VACUUM

References

Book	Author	Why
"PostgreSQL 14 Internals"	Egor Rogov	Free — covers pages, heap, B-Tree, VACUUM internals
"SQL Performance Explained"	Markus Winand	Best practical index guide — use-the-index-luke.com
"Designing Data-Intensive Applications"	Martin Kleppmann	B-Trees, LSM-Trees, storage engine theory
"Database Internals"	Alex Petrov	Deep dive into B-Tree structure and page layout

Online: [PostgreSQL Docs — Indexes](#) | [Redgate — What Is a Database Index?](#)

Core Principle: An index converts $O(N)$ full-table scans into $O(\log N)$ targeted lookups — at the cost of extra writes, storage, and maintenance.